

Day 1 — REST API Design Principles & Resource Modeling

Project Domain: Library Management System

1. API Overview

This API is designed to manage a library system where users can browse books, manage members, and track book borrowings.

The API follows REST principles by:

- Modeling the system using resources (nouns).
- Using HTTP methods to represent operations.
- Maintaining stateless communication.
- Returning standard HTTP status codes.

2. Core Resources

The main resources in the system are:

Resource	Description
Books	Represents available books in the library
Members	Represents library members
Authors	Represents book authors
Borrowings	Represents book borrowing records

All resources use plural naming conventions:

- /api/v1/books
- /api/v1/members
- /api/v1/authors
- /api/v1/borrowings

3. REST Resource Endpoints

Books Resource

Get All Books

GET /api/v1/books

Description:

- Returns a list of all available books.

Success Response:

200 OK

Error Example:

500 Internal Server Error

Get Book By ID

GET /api/v1/books/{id}

Description:

- Returns details of a specific book.

Success Response:

200 OK

Error Response:

404 Not Found

When the requested book does not exist.

Create New Book

POST /api/v1/books

Description:

- Creates a new book.

Success Response:

201 Created

Error Response:

400 Bad Request

When the provided data is invalid.

Update Book

PUT /api/v1/books/{id}

Description:

- Updates an existing book.

Success Response:

200 OK

Error Response:

404 Not Found

When the book does not exist.

Delete Book

DELETE /api/v1/books/{id}

Description:

- Deletes a book from the system.

Success Response:

204 No Content

Error Response:

404 Not Found

When the book does not exist.

4. Nested Resource Example

Member Borrowings

A member can have multiple borrowing records.

Endpoint:

GET /api/v1/members/{id}/borrowings

Description:

- Returns all borrowing records belonging to a specific member.

Example:

GET /api/v1/members/5/borrowings

Success Response:

200 OK

Error Response:

404 Not Found

When the member does not exist.

5. HTTP Status Code Usage

Status Code	Usage
200 OK	Successful GET or PUT request
201 Created	Successful resource creation
204 No Content	Successful deletion without response body
400 Bad Request	Invalid input data
401 Unauthorized	User authentication required
403 Forbidden	User does not have permission
404 Not Found	Resource does not exist
500 Internal Server Error	Unexpected server error

6. API Versioning Strategy

The API uses URL-based versioning.

Format:

/api/v1/{resource}

Examples:

GET /api/v1/books

GET /api/v1/members

POST /api/v1/books

Reason:

- Makes API versions clear.
 - Allows future updates without breaking existing clients.
 - Easy for developers to understand and maintain.
-

7. REST Design Rules Followed

The API follows these REST conventions:

- Resources are represented as plural nouns.
 - HTTP methods define actions.
 - URLs do not contain verbs.
 - Standard HTTP status codes are returned.
 - API versioning is included from the beginning.
 - Resources are connected through meaningful relationships.
-

8. Postman Collection Structure

Collection Name:

Library Management API

Requests:

GET /api/v1/books

GET /api/v1/books/{id}

POST /api/v1/books

PUT /api/v1/books/{id}

DELETE /api/v1/books/{id}

GET /api/v1/members/{id}/borrowings

Conclusion

This REST API design provides a consistent and scalable structure that follows REST principles and can be implemented later using ASP.NET Core Web API.